

Adaptive Camera Cleaning System

A plain-language walkthrough of what it is, what I built, and how every part works, written to re-learn the project from scratch. UIUC ECE 445 senior design, Jan to May 2025, team of three. Repo: github.com/dbhan08/Weather-Resistant-Camera

1. The one-paragraph version

Cameras outdoors get rain and frost on the lens and go blind. We built a box that fixes that by itself. Inside is a tiny computer chip (an STM32 microcontroller) with three jobs. It looks at the lens with a small camera and runs a neural network to decide if there is water on it. If there is, it moves a wiper across the lens with a servo motor. Separately, it reads a temperature sensor and turns on a heating pad when it gets cold, to melt frost. No laptop, no person. Plug it in and it looks after itself.

The one thing to remember: the whole product is a loop that runs once a second on the chip: read temperature, grab a picture, ask the neural network 'is the lens wet?', and move the wiper or switch the heater based on the answers.

Result	Value
Detection accuracy on our 100-image test set	89 %
When the wiper fires	5 % or more of the picture is classed as water
When the heater fires	temperature below the threshold (20.5 C in the demo)
Outdoor test	2 hours continuous
Size of the neural network on the chip	about 20,000 numbers, stored as 8-bit integers

2. What I personally did

Three people. The project was split into seven pieces: heater, temperature sensor, microcontroller, wiper, power, camera, and vision (the software that decides if the lens is wet). I owned the last two and the glue between them and the chip.

- **Camera.** Picked the camera modules, got a live picture out of them, and wrote the code that pulls a frame into the chip's memory.
- **Vision.** Wrote the rain detector. First a quick hand-tuned version for bring-up, then the real one: a neural network that I built from a published model, adapted to our lens, shrank to fit the chip, and got running on the chip.
- **Integration.** Wrote the main loop that ties temperature, camera, neural network, wiper and heater together, plus the serial-cable protocol we used during development and the logging used to score the test set.
- **Writing.** Introduction, Problem Statement and Proposed Solution in the proposal and design doc. The Requirements and Verification section of the final paper, with all the test data.
- **Shared.** Circuit board layout, machine-shop visits for the enclosure, soldering.

3. The parts, and what each one physically does

3.1 The microcontroller (STM32F4)

A microcontroller is a whole small computer on one chip: processor, memory, and a set of built-in helper circuits called peripherals. Ours is an STM32F4 made by ST. It has 512 KB of flash (permanent storage where the program and the neural network live) and 192 KB of SRAM (working memory that is wiped at power-off). Those two numbers drove almost every design decision: the neural network and one camera frame both had to fit in 192 KB, together.

The peripherals we use, each explained where it comes up: a timer for the servo, an ADC for the temperature sensor, a UART for the serial cable, an I2C bus and a DCMI port for the camera, and a DMA engine to move camera data without the processor.

We designed our own circuit board for it in KiCad. The final demo ran the same program on a Nucleo, which is ST's ready-made board with the same chip, because we destroyed the chip on our second board while hand-soldering it. The chip's pins are 0.5 mm apart, and a hot iron on that pitch is unforgiving.

3.2 The wiper: a servo on a PWM timer

A servo is a small motor that goes to an angle you ask for and holds it. You tell it the angle with a pulse: send a short electrical pulse every 20 milliseconds, and the width of the pulse is the angle. About 1 ms means one end, about 2 ms means the other end. This kind of signal is called PWM, pulse width modulation.

The chip's timer peripheral (TIM2) makes that signal for us. We set it up once so that one full 20 ms period equals some number of timer ticks, and then we just write a 'compare' value that says how many ticks the pulse should stay high. Bigger compare value, wider pulse, bigger angle. The two ends of our servo came out at 210 ticks and 1050 ticks, so the code maps 0 to 180 degrees onto that range with one line of arithmetic:

```
void Set_Servo_Angle(TIM_HandleTypeDef *htim, uint32_t ch, uint8_t angle) {
    uint32_t pulse = 210 + (angle * (1050 - 210) / 180); // degrees -> ticks
    __HAL_TIM_SET_COMPARE(htim, ch, pulse);              // timer does the rest
}
```

A wipe is a sweep from 60 degrees to 115 and back, moving 3 degrees every 100 ms. Small steps with a pause make the arm move smoothly instead of slamming into the housing. The whole sweep takes about 3.7 s.

```
for (uint8_t a = 60; a <= 115; a += 3) { Set_Servo_Angle(&htim2, TIM_CHANNEL_1, a); HAL_Delay(100); }
for (uint8_t a = 115; a > 60; a -= 3) { Set_Servo_Angle(&htim2, TIM_CHANNEL_1, a); HAL_Delay(100); }
```

3.3 The temperature sensor: an ADC reading

The STM32 has a temperature sensor built into the silicon. It outputs a voltage that rises with temperature. To read a voltage in software you need an ADC, an analog-to-digital converter, which turns a voltage into a number. Ours is 12-bit, so it gives a number from 0 to 4095.

To turn that number into degrees you need two known points. ST measures every chip at the factory at 30 C and 110 C and burns the two ADC readings into a fixed spot in memory. So the conversion is a straight line through those two points. We also found our readings ran 9 C high against a real thermometer, so we subtract 9.

```
#define TS_CAL1 (*((uint16_t*)0xFF7A2C)) // factory reading at 30 C
#define TS_CAL2 (*((uint16_t*)0xFF7A2E)) // factory reading at 110 C
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY); // wait for the number
raw = HAL_ADC_GetValue(&hadc1);
t = (raw - TS_CAL1) * (110.0f - 30.0f) / (TS_CAL2 - TS_CAL1) + 30.0f - 9.0f;
```

3.4 The heater: a MOSFET switch on one pin

The heating pad needs more current than a chip pin can supply, and it runs at 5 V while the chip runs at 3.3 V. So the chip does not power the heater. It flips a switch. The switch is a MOSFET, a transistor that lets current flow between two of its legs when a voltage is put on the third leg (the gate). Between the chip and the gate sits a level shifter, a small part that turns the chip's 3.3 V signal into the 5 V the gate wants.

Our wiring is inverted: the pad is on when pin PA5 is low. Pulling the pin low removes the voltage that was keeping the MOSFET off. An LED next to it lights when the heater is on so you can see the state.

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, temp_c < HEATER_THRESH_C ? GPIO_PIN_RESET : GPIO_PIN_SET);
```

The threshold is 20.5 C in the demo code, not 0 C. We tested indoors with a cooler of ice, which cools the sensor to the high teens but never to freezing, so we raised the threshold to show the behaviour. In the field you set it to 1 or 2 C.

Design rule: the heater only ever listens to the temperature sensor. The camera and neural network never touch it. Lots of things look like frost in a picture. Temperature is unambiguous.

3.5 Power

One 5 V regulator feeds the servo and the heater. A 3.3 V regulator feeds the chip. A regulator is a part that takes a messy input voltage and gives a steady fixed output. We verified both rails with a meter before soldering anything else, so a power fault could not be confused for a code bug later.

3.6 The serial cable (UART) for development and logging

A UART sends bytes one at a time down two wires. The chip's USART2 runs at 115200 bits per second and shows up as a serial port on a laptop. During development the laptop sent the word START to run the wiper. In the final product the cable is optional: the chip prints one status line per second on it, and that log is how we scored the test set.

4. The camera: getting a picture into the chip

The camera is an OV7670, a fingernail-sized sensor that costs a few dollars. It does not send JPEG files. It streams raw pixels, one after another, on eight data wires, with a clock wire that ticks once per pixel and two more wires that say 'new row' and 'new frame'. Reading that with the processor would eat all its time, so the STM32 has a peripheral for it, DCMI (digital camera interface). DCMI grabs each pixel as it arrives and hands it to DMA, a memory-copy engine that writes into SRAM with no processor involved. The processor starts the capture, goes off to do other work, and gets told when the frame is done.

Before any of that, the camera has to be configured. It has a few dozen internal settings registers written over a two-wire bus called SCCB, which is the same as I2C, a standard two-wire chip-to-chip bus. The settings we write: reset, colour mode, output format, and 'shrink the picture by 4 in each direction'.

```
static const uint8_t k_ov7670_init[][2] = {
    {0x12, 0x80}, // reset
    {0x12, 0x14}, // colour (RGB) mode
    {0x40, 0xD0}, // RGB565 pixel format
    {0x0C, 0x04}, // enable the down-scaler
    {0x72, 0x22}, // shrink by 4 in x and y -> 160 x 120
    {0x13, 0xE7}, // auto gain, auto white balance, auto exposure
    ...
};
HAL_I2C_Master_Transmit(&hi2c1, 0x42, {reg, val}, 2, 100); // one register per call
```

4.1 Why 160 by 120 and what RGB565 means

The camera can do 640 by 480, but that frame would be 600 KB, three times all our SRAM. We shrink to 160 by 120. Each pixel comes as RGB565: 16 bits per pixel, with 5 bits of red, 6 of green, 5 of blue. That is 38 KB for a frame. The neural network wants normal 8-bit-per-colour pixels (RGB888), so after capture we expand each pixel from 16 bits to 24. The trick in the expansion is to copy the top bits into the empty bottom bits so that full white comes out as 255, not 248.

```
HAL_DCMI_Start_DMA(&hdcmi, DCMI_MODE_SNAPSHOT, (uint32_t)s_frame565, (CAM_W*CAM_H*2)/4);
while (!s_frame_done) // set by the frame-complete interrupt
    if (HAL_GetTick() - t0 > 500) return 0; // camera unplugged -> give up
r5 = (p >> 11) & 0x1F; g6 = (p >> 5) & 0x3F; b5 = p & 0x1F;
rgb888[3*i+0] = (r5 << 3) | (r5 >> 2); // 5 bits -> 8 bits
rgb888[3*i+1] = (g6 << 2) | (g6 >> 4);
rgb888[3*i+2] = (b5 << 3) | (b5 >> 2);
```

Memory so far: 38 KB raw frame plus 58 KB expanded frame. That leaves room for the 48 KB the neural network needs, and the rest for the program's own variables and stack.

5. The vision: how the chip decides the lens is wet

5.1 The idea: cut the picture into tiles and grade each tile

We do not try to outline each drop. We cut the 160 by 120 picture into a grid of 48 by 48 pixel squares, a 3-wide by 2-high grid of six tiles, and ask one question per tile: 'water, or clean?' The fraction of tiles that say water is the coverage. Two of six tiles wet is 33 percent coverage. The wiper fires at 5 percent or more, which for a 6-tile grid means any single tile.

This is a known approach from self-driving-car cameras (Valeo's SoilingNet papers), where it is called a coverage metric. It is cheap enough to run on a microcontroller because the network looks at one small tile at a time, and the grid also tells you where on the lens the water is.

5.2 What a neural network is, in this context

A convolutional neural network (CNN) is a function that takes an image in and gives a score per class out. Inside it is a stack of layers. Each layer slides small filters across the image and produces a new, smaller image of 'features': edges first, then blobs, then things like 'a bright ring with a dark edge', which is what a water drop looks like. The numbers in the filters are the weights, and training means adjusting the weights until the scores are right on thousands of labelled example tiles. Once trained, running the network is just multiplications and additions, which a microcontroller can do.

5.3 Step 1: start from a published model instead of from zero

Training from zero needs thousands of hand-labelled pictures of our lens, which we did not have. So I started from a model somebody had already trained on rain: Breckon's group at Durham published a small network in 2018 that looks at 30 by 30 patches from a car windshield camera and says drop or no drop, with 95 percent accuracy. They released the trained weights under an MIT licence.

Problem: the weights were saved in TensorFlow 1 with a library called TFLearn, both of which are dead. Nothing modern can load them. But the file is just a list of named arrays of numbers, and TensorFlow 2 can still read the raw arrays. So I rebuilt the same network layer by layer in Keras (the modern TensorFlow API) and copied each array into the matching layer by name, checking every shape:

```
_NAME_MAP = {"conv1": "Conv2D", "conv2": "Conv2D_1", ..., "fc3": "FullyConnected_2"}
reader = tf.train.load_checkpoint(tfl_path)      # opens the old TF1 file
w = reader.get_tensor(f"{scope}/w")             # weights array
b = reader.get_tensor(f"{scope}/b")             # bias array
assert w.shape == layer.get_weights()[0].shape # fail loudly if the port is wrong
layer.set_weights([w, b])
```

The network itself: five convolution layers (96 filters of 11x11, then 256 of 5x5, then 384, 384 and 256 of 3x3), with pooling layers between that shrink the image, then two dense layers of 4096 numbers each, then a 2-way output. To check the port, I ran the 16 sample patches that ship with their code and compared the answers.

5.4 Step 2: adapt it to our lens (fine-tuning)

Their camera looked through a windshield a metre away. Ours is millimetres from the drops, so our drops are huge and blurry by comparison. A model trained on one does worse on the other. The fix is fine-tuning: keep the trained weights, then train a little more on pictures like ours so it adjusts.

For pictures like ours I used a public dataset, RaindropsOnWindshield: 8,190 frames where every drop has been outlined by hand (a mask), free to use with credit. A script cuts it into 30 by 30 patches. A patch counts as 'water' if at least 30 percent of it is inside a drop outline. It counts as 'clean' only if the patch and a one-patch border around it have no drop at all, so a drop edge never sneaks into a 'clean' example and confuses the training. Frames from our own camera were added the same way. The split into training and test sets is by video clip, so the test never contains frames from a clip the model trained on, which would inflate the score.

```
# data_windshield.py -- cut labelled 30x30 patches from the mask images
# positive: centred on a drop pixel, mask covers >= 30 % of the patch
y0, x0 = drop_y - PATCH // 2, drop_x - PATCH // 2
if mask[y0:y0+PATCH, x0:x0+PATCH].mean() >= 0.30:
    pos.append(img[y0:y0+PATCH, x0:x0+PATCH])
# negative: random spot, and the patch PLUS a one-patch border has no drop at all
m = mask[max(0, y0-PATCH):y0+2*PATCH, max(0, x0-PATCH):x0+2*PATCH]
if m.max() == 0:
    neg.append(img[y0:y0+PATCH, x0:x0+PATCH])
# split by video clip, not by frame
```

```

rng.shuffle(sequences)
train, val, test = sequences[:70%], sequences[70%:85%], sequences[85%:]
# -> data/patches/{train,val,test}/{raindrop,non_raindrop}/*.png

```

The fine-tuning itself is the standard recipe. First freeze the convolution layers and train only the dense layers at the end at a normal learning rate for a few rounds. Then unfreeze everything and train at a tiny learning rate, stopping as soon as the score on held-out data stops improving. Each training image is randomly flipped and rotated up to 45 degrees so the model does not memorise orientation.

```

# finetune.py
aug = keras.Sequential([RandomFlip("horizontal_and_vertical"), RandomRotation(0.125), # +-45 deg
                        RandomBrightness(0.15), RandomContrast(0.15)])

# stage 1: only the dense head learns
for layer in model.layers:
    layer.trainable = layer.name.startswith("fc")
model.compile(keras.optimizers.Adam(1e-3), "categorical_crossentropy", metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=5)

# stage 2: everything learns, very gently, stop when validation stops improving
for layer in model.layers:
    layer.trainable = True
model.compile(keras.optimizers.Adam(1e-5), "categorical_crossentropy", metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10,
        callbacks=[EarlyStopping(patience=3, restore_best_weights=True)])
test = model.evaluate(test_ds, return_dict=True) # written to finetune_report.json

```

5.5 Step 3: shrink it 900 times (distillation)

Here is the wall. Count the weights: those two dense layers are 256 by 4096 and 4096 by 4096. Together with the rest that is about 18 million numbers. In 4-byte floats that is 75 MB. Even squeezed to 1 byte each it is 19 MB. Our chip has 512 KB of flash. The big model cannot go on the chip, full stop. It also uses one layer type (local response normalization) that the chip-side library does not support.

So the big model does not go on the chip. It becomes the teacher. I designed a tiny student network that does fit, and trained the student to copy the teacher. This is knowledge distillation. Instead of only telling the student 'this tile is water', you show it the teacher's full opinion, like '80 percent water, 20 percent clean'. Those soft answers carry much more information per example, including what the teacher learned about hard cases like glare, which is exactly where our first detector failed.

The student: input is a 48 by 48 tile. A first small convolution, then five 'depthwise separable' blocks, a MobileNet trick that does a cheap per-channel filter followed by a cheap 1x1 mixing filter instead of one expensive full filter, which cuts the maths by about 8 times. Then average everything down to one vector and a final layer that outputs two scores. About 20,000 weights total. Every layer type is one the chip-side library supports.

```

x = conv3x3_stride2(inp, 16) # 48x48 -> 24x24
x = ds_block(x, 32, stride=2); x = ds_block(x, 32, 1) # -> 12x12
x = ds_block(x, 64, stride=2); x = ds_block(x, 64, 1) # -> 6x6
x = ds_block(x, 128, stride=2) # -> 3x3
x = GlobalAveragePooling2D()(x) # 3x3x128 -> 128 numbers
out = Dense(2)(x) # score for clean, score for water

# the distillation loss, per batch:
loss = 0.7 * KL(teacher_soft, student_soft) * T**2 + 0.3 * CrossEntropy(true_label, student)
# T = 4 'softens' the teacher's answers so the small differences show through

```

5.6 Step 4: turn floats into 8-bit integers (quantization)

Trained weights are 32-bit floating point numbers. Microcontrollers are slow at floating point and flash is small, so we convert the whole network to 8-bit integers. Each weight becomes a number from -128 to 127 plus one shared scale factor per layer that says what one step is worth. Same for the numbers flowing between layers at run time. That is 4 times smaller and several times faster, and on a network this size it costs almost no accuracy.

To pick the scale factors for the run-time numbers, the converter needs to see typical inputs. You hand it a 'representative dataset', a few hundred real tiles, and it runs them through and records how big each layer's numbers get. I use tiles from the validation set, not the training set, so the scales are not tuned to the exact images the weights were fitted on. The output is a .tflite file, plus that file dumped as a C array so it compiles straight into the firmware.

```
conv = tf.lite.TFLiteConverter.from_keras_model(student)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
conv.representative_dataset = lambda: ([x[None]],) for x in val_tiles)
conv.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
conv.inference_input_type = conv.inference_output_type = tf.int8
open("raindrop_student_int8.tflite", "wb").write(conv.convert())

# write_c_array(): the .tflite bytes become a normal C source file
body = "".join(f"0x{b:02x}" for b in model_bytes)
write(f"alignas(16) const unsigned char g_raindrop_student[] = {{{body}}};")
write(f"const unsigned int g_raindrop_student_len = {len(model_bytes)};")
```

How that array ends up in flash: on a microcontroller, anything declared *const* is placed by the linker in the read-only data section, and that section lives in the flash address range. So compiling `raindrop_student_int8.cc` into the firmware and flashing the board puts the weights in flash with the rest of the program. No copying code is needed. At run time TensorFlow Lite Micro reads the model straight from that address (`tflite::GetModel(g_raindrop_student)` returns a pointer, not a copy), so the weights never take up any SRAM. Only the 48 KB working arena does. The `alignas(16)` is there because the library reads the model with 16-byte aligned loads.

5.7 Step 5: run it on the chip (TensorFlow Lite Micro)

TensorFlow Lite Micro (TFLM) is a small C++ library that runs .tflite models on microcontrollers with no operating system and no dynamic memory. You give it the model bytes, a list of which layer types to compile in (we register only the six we use, to save flash), and a fixed block of memory called the arena where it keeps all the intermediate results. Ours is 48 KB. At boot:

```
g_model = tflite::GetModel(g_raindrop_student);           // point at the array in flash
g_resolver.AddConv2D(); g_resolver.AddDepthwiseConv2D(); g_resolver.AddMean();
g_resolver.AddFullyConnected(); g_resolver.AddReshape(); g_resolver.AddSoftmax();
static tflite::MicroInterpreter interp(g_model, g_resolver, g_arena, 48 * 1024);
interp.AllocateTensors();                                 // carve up the arena once
g_in = interp.input(0); g_out = interp.output(0);
```

Per frame, the code walks the 3 by 2 grid. For each tile it copies the 48 rows out of the big frame into a small buffer, converts each pixel byte into the network's int8 input using the scale and zero-point that the converter stored inside the model, runs the network, and reads the two output scores. Whichever score is bigger wins; there is no need to convert back to real probabilities, because the comparison works directly on the integers.

```
// the network was trained on pixel/255, so:
q = round((pixel / 255) / input_scale) + input_zero_point;  clamp to [-128, 127]
g_in->data.int8[i] = q;
...
g_interp->Invoke();
raindrop = g_out->data.int8[1] > g_out->data.int8[0];
```

```
hits += raindrop;  
return (float)hits / (rows * cols);    // coverage, 0.0 to 1.0
```

A hardware cycle counter inside the processor (the DWT) times each pass. The count is printed on the status line, which is where our per-frame timing numbers came from.

6. Putting it together: the main loop

Everything above is called from one loop that runs once per second. Read it top to bottom and you have the whole product.

```
for (;;) {
    uint32_t t0 = HAL_GetTick();

    // 1. temperature -> heater
    float temp_c = ReadTemperatureC();
    int heat = temp_c < HEATER_THRESH_C;
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, heat ? GPIO_PIN_RESET : GPIO_PIN_SET);

    // 2-3. picture -> neural network -> coverage
    float cover = -1.0f;
    if (cam_ok && nn_ok && CameraCaptureRGB888(s_frame))
        cover = RaindropCoverage(s_frame, CAM_W, CAM_H, s_grid);

    // 4. coverage -> wiper, with two safety rules
    if (cover >= WIPE_THRESHOLD && temp_c > 0.0f && HAL_GetTick() - last_wipe > WIPE_COOLDOWN_MS) {
        Wipe();
        last_wipe = HAL_GetTick();
    }

    // 5. one line of log per second
    printf("T=%.1fC heat=%d cover=%.2f grid=%s nn_us=%lu\r\n", temp_c, heat, cover, grid, nn_us);
    HAL_Delay(LOOP_PERIOD_MS - (HAL_GetTick() - t0));
}
```

The two safety rules on the wiper: it never runs at or below 0 C, because a wiper dragging over ice strips the blade and stalls the servo, and it never runs again within 4 seconds, because one sweep takes 3.7 seconds and the next frame would still see the wet lens mid-wipe and fire again.

If the camera or the model fails to start, the loop keeps running the heater and prints the failure. During development it could also fall back to the laptop sending START over the cable.

6.1 Where the memory goes

Buffer	Size	Why
Raw RGB565 camera frame	38 KB	DMA writes here
Expanded RGB888 frame	58 KB	what the network reads
TFLM arena	48 KB	all intermediate layer outputs
Model (in flash, not SRAM)	about 25 KB	the int8 weights
Total SRAM	about 150 KB of 192 KB	rest is stack and HAL state

6.2 Building it

In STM32CubeIDE, turn on DCMI, a DMA stream for it, I2C1, and a clock output for the camera in the .ioc file and let CubeMX generate the setup code. Pull the TensorFlow Lite Micro sources in as a folder (there is a script in their repo, create_tflm_tree.py, that produces exactly the files you need) and compile them as C++17. Add the three firmware files and the generated model array. Swap the old main.c for main_raindrop.c. Flash and open the serial port.

7. How we got here: the earlier detector and what it taught us

The first detector was not a neural network. It was a quick OpenCV pipeline on the laptop, written to get something moving. OpenCV is a standard image library. The idea: water on glass looks grey, so convert the picture to HSV (hue, saturation, value: colour, how colourful, how bright), keep pixels that are not colourful (saturation under 40) and not dark (value over 100), fill the little gaps to make solid blobs, draw circles around the biggest blobs, and count what fraction of the picture is inside them. Over 5 percent, send START down the cable.

```
_, s_mask = cv2.threshold(s, 40, 255, cv2.THRESH_BINARY_INV) # low saturation
_, v_mask = cv2.threshold(v, 100, 255, cv2.THRESH_BINARY) # not dark
closed = cv2.morphologyEx(cv2.bitwise_and(s_mask, v_mask), cv2.MORPH_CLOSE, kernel)
contours = sorted(cv2.findContours(closed, ...)[0], key=cv2.contourArea, reverse=True)
for c in contours[1:3]: cv2.circle(...) # skip the biggest (lens ring), fill next two
if black_percent > 5: ser.write(b"START")
```

It worked on clean test pictures and failed in the enclosure: glare, scratches and surface texture all got flagged as water. Our TA found the real reason by looking at the hardware, not the code: the camera is so close to the surface where drops form that it is effectively zoomed way in, and tiny texture becomes big blobs. No threshold fixes that. You need something that has learned what a drop looks like versus what glare looks like. That is the whole reason for the neural network path, and why distillation from a teacher that had seen lots of glare mattered.

The 5 percent rule survived from the heuristic into the final product. Everything else was replaced.

8. How we tested it

- **Detection accuracy: 89 percent.** 100 test images of rain and ice on the lens in different lighting. For each, the chip's printed tile grid was compared with what was actually on the lens.
- **Each part alone first.** Regulators on a meter (5 V and 3.3 V). MOSFET output with and without gate drive. Servo on a servo tester at the machine shop, which is how we found one motor was simply dead. Temperature against a real thermometer, which gave us the 9 C offset. PWM on a scope.
- **The model pipeline.** Every stage writes a report file: did every weight array match on the port, how did the sample patches score, test accuracy after fine-tuning, student test accuracy after distillation, and how often int8 agrees with the float model after quantization. A separate PyTorch pipeline in the repo (lens_soiling/) has nine automated tests as a fallback.
- **Everything together.** Final demo with all subsystems triggering each other: an ice cooler and the 20.5 C threshold for the heater, water on the lens for the wiper, LED confirming heater state. Then two hours outdoors.

9. What went wrong and what we did

Problem	Why	What we did
First detector flagged glare as water	Lens millimetres from the wet surface magnifies texture	Replaced it with a learned tile classifier; distilled from a teacher that had seen glare
Published model far too big for the chip	18 M weights; uses a layer TFLM lacks	Kept it as the teacher; distilled a 20 k-weight student
Published weights would not load	Saved by dead TF1 / TFLearn	Read the raw arrays, rebuilt the network in Keras, copied by name

Frame plus model did not fit memory	192 KB SRAM	160x120 frames, 48 KB arena, only six layer types compiled in
Second PCB's chip destroyed	0.5 mm pin pitch, hand soldering	Ran the same firmware on the Nucleo dev board
Servo would not move	Dead motor	Proved it on a servo tester, replaced it
Could not test freezing indoors	Ice cooler tops out around 15 C	Raised the heater threshold to 20.5 C for the demo

10. If someone asks

- **Why tiles instead of finding each drop?** Cheaper by far, fits a microcontroller, and the wipe decision only needs 'how much', not 'what shape'.
- **Why not train your own small model from scratch?** No big labelled dataset of our lens. The teacher had already learned rain and glare from thousands of images; distillation moves that into the student for free.
- **Why does the heater ignore the camera?** Frost, fog, and dirt look alike. Temperature does not lie.
- **Hardest bug?** The glare false positives. The fix came from looking at the mechanical setup, not the code.
- **What would you change?** Mount the camera further from the wet surface. Add a frost class to the student. Use an STM32H7 with more memory for a full-size frame and a bigger student.

References: Guo, Akcay, Adey, Breckon, ICIP 2018 (github.com/tobybreckon/raindrop-detection-cnn). Soboleva & Shipitko, RaindropsOnWindshield, 2021 (Zenodo 4680442). Uricar et al., SoilingNet, ITSC 2019. Hinton, Vinyals, Dean, Distilling the Knowledge in a Neural Network, 2015. Howard et al., MobileNets, 2017. Code: Weather-Resistant-Camera/CNN/ and main.c.